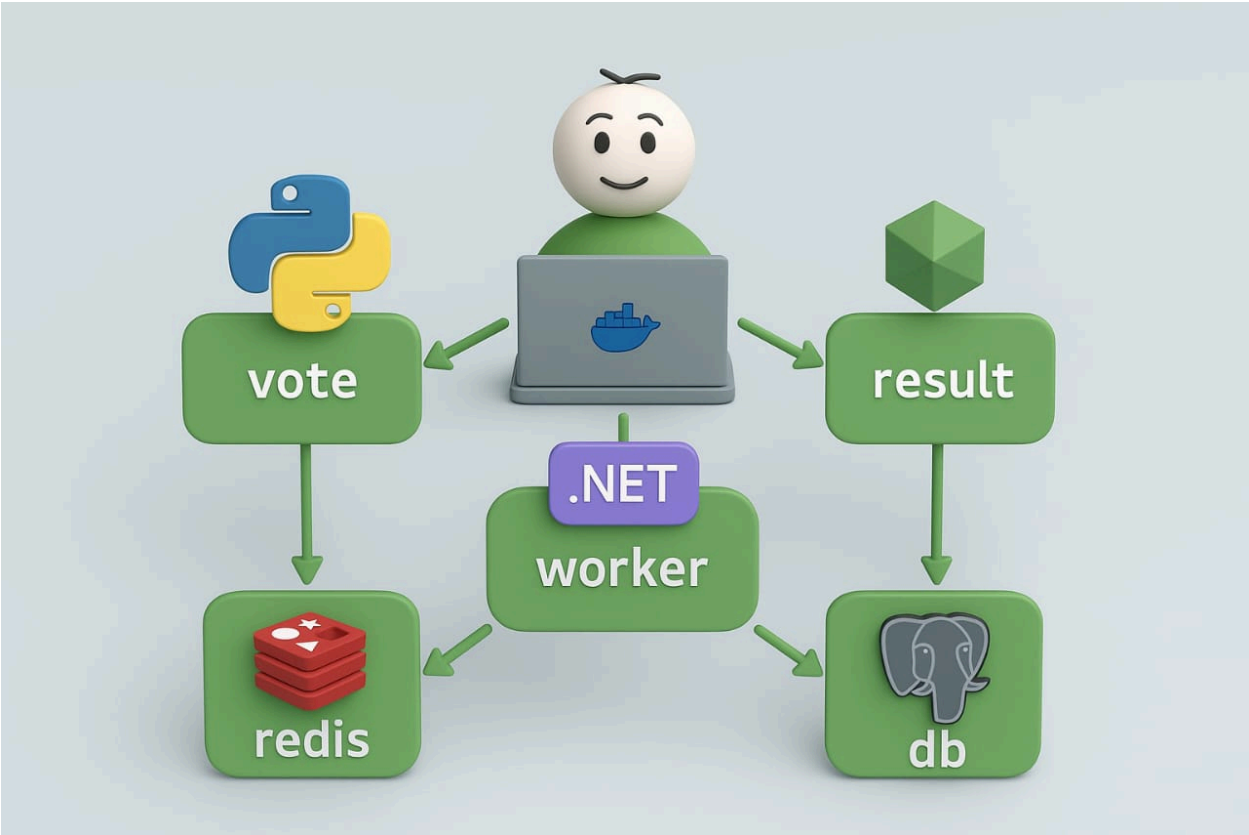
5-Tier Voting-Application Deployment Using Docker



Voting App (Flask → Redis → .NET Worker → Postgres → Node.js Results).

⚙️ Architecture:-

1. Vote App (Python/Flask – Frontend)

- Simple web app where users vote for Cats 😸 or Dogs 🐶.
- Collects input and sends it to Redis.

2. Redis (In-Memory Cache)

- Fast, temporary storage of votes.
- Handles spikes in user activity.

3. Worker (.NET Service – Processor)

- Continuously pulls votes from Redis.
- Inserts them into Postgres for permanent storage.

4. Postgres (SQL Database – Storage)

- Stores all votes **permanently**.
- Uses a **volume** so data persists even if the container restarts.

5. Result App (Node.js – Results Dashboard)

- Web app that queries Postgres.
- Displays live voting results to users in real-time.

Step1:- Clone the repo *example-voting-app* from GitHub.

```
controlplane:~$ git clone https://github.com/dockersamples/example-voting-app
Cloning into 'example-voting-app'...
remote: Enumerating objects: 1179, done.
remote: Total 1179 (delta 0), reused 0 (delta 0), pack-reused 1179 (from 1)
Receiving objects: 100% (1179/1179), 1.21 MiB | 3.21 MiB/s, done.
Resolving deltas: 100% (449/449), done.
controlplane:~$ ls
example-voting-app  filesystem
controlplane:~$ cd example-voting-app/
controlplane:~/example-voting-app$ ls
LICENSE      architecture.excalidraw.png  docker-stack.yml    result     worker
MAINTAINERS  docker-compose.images.yml    healthchecks        seed-data
README.md    docker-compose.yml           k8s-specifications  vote
controlplane:~/example-voting-app$
```

Step2:- Create a Docker network named "votingapp" to enable communication between the application's services.

```
controlplane:~/example-voting-app$ docker network create votingapp
41ae492f004e3cb25f757f8eb22f259e6c4e80c49fb4ee5296c55da2d1ecc0c3
controlplane:~/example-voting-app$
```

Step3:- Initialize a persistent Docker volume `db-data` for PostgreSQL storage.

```
controlplane:~/example-voting-app$ docker volume create db-data
db-data
controlplane:~/example-voting-app$
```

Step4:- Step 3: Start Redis (Message Queue)

```
controlplane:~/example-voting-app$ docker run -d --name=redis --network=votingapp redis:alpine
Unable to find image 'redis:alpine' locally
alpine: Pulling from library/redis
9824c27679d3: Pull complete
9880d81ff87a: Pull complete
168694ef5d62: Pull complete
f8eab6d4856e: Pull complete
1f79dac8d2d4: Pull complete
4f4fb700ef54: Pull complete
61cfb50eeff3: Pull complete
Digest: sha256:987c376c727652f99625c7d205a1cba3cb2c53b92b0b62aade2bd48ee1593232
Status: Downloaded newer image for redis:alpine
fe79a84c9fb3b1af6fbc38d91ae6bf1e676ce0fa29ffb253bb2aa34965fdc2ea
controlplane:~/example-voting-app$
```

Step5:- Start Postgres (Persistent Database)

```
controlplane:~/example-voting-app$ docker run -d --name=db --network=votingapp -e POSTGRES_USER=postgres -e POSTGRES_P
ASSWORD=postgres -v db-data:/var/lib/postgresql/data -p 5432:5432 postgres:15-alpine
Unable to find image 'postgres:15-alpine' locally
15-alpine: Pulling from library/postgres
9824c27679d3: Already exists
45222bf54768: Pull complete
cc455ce33b7a: Pull complete
6e59367d2e76: Pull complete
a9186699d5f8: Pull complete
90793d70c9dc: Pull complete
d97eb1910934: Pull complete
8a4d728ab743: Pull complete
cb75953ddf8d: Pull complete
4c5335ab75e4: Pull complete
02c858cee38f: Pull complete
Digest: sha256:dfcf0459185089e88a43197975780f5a3078acd5ece84824a14c9d6fbbab02d0
Status: Downloaded newer image for postgres:15-alpine
405a67cac9c43046dfef55679b308ff6268f2750ec793422fa9482c936ca2820
controlplane:~/example-voting-app$
```

Step6:- Step 5: Build Service Images (vote, result, worker)

```
controlplane:~/example-voting-app/vote$ docker build -t voteimage .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   12.8kB
Step 1/13 : FROM python:3.11-slim AS base
3.11-slim: Pulling from library/python
ce1261c6d567: Pull complete
11b89692b208: Pull complete
764e05fe66b6: Pull complete
a4aefcec16c5: Pull complete
Digest: sha256:a0939570b38cddeb861b8e75d20b1c8218b21562b18f301171904b544e8cf228
```

Step7:- Build Result Service (Node.js)

```
controlplane:~/example-voting-app/result$ docker build -t resultimage .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  291.3kB
Step 1/11 : FROM node:18-slim
18-slim: Pulling from library/node
61320b01ae5e: Pull complete
b98d3ae1ab80: Pull complete
b1831021e35a: Pull complete
c768ab8cba73: Pull complete
8c994cf49dd1: Pull complete
Digest: sha256:f9ab18e354e6855ae56ef2b290dd225c1e51a564f87584b9bd21dd651838830e
```

Step8:- Build Worker Service

```
controlplane:~/example-voting-app/worker$ docker build -t workerimage .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  10.24kB
Step 1/14 : FROM --platform=${BUILDPLATFORM} mcr.microsoft.com/dotnet/sdk:7.0 AS build
failed to parse platform : "" is an invalid OS component of "": OSAndVersion specifier component must match "^([A-Za-z
0-9_-]+)(?:\\(([A-Za-z0-9_.-]*)\\))?$": invalid argument
controlplane:~/example-voting-app/worker$
```

The build may fail with `--platform=${BUILDPLATFORM}` error; to fix this, use Docker Buildx and download it from the official Docker documentation.

- **curl -SL https://github.com/docker/compose/releases/download/v2.39.3/docker-compose-linux-x86_64 -o /usr/local/bin/docker-compose**
- **chmod +x /usr/local/bin/docker-compose**

```
controlplane:~$ curl -SL https://github.com/docker/compose/releases/download/v2.39.3/docker-compose-linux-x86_64 -o /u
sr/local/bin/docker-compose
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100 72.8M  100 72.8M    0     0  14.5M      0  0:00:05  0:00:05 --:--:-- 17.7M
controlplane:~$ chmod +x /usr/local/bin/docker-compose
```

Use `docker buildx build -t worker-img .` to build the worker image with Buildx.

```
controlplane:~/example-voting-app/worker$ docker buildx build -t worker-img .
[+] Building 11.2s (5/15)                                                                          docker:default
 => [stage-1 1/3] FROM mcr.microsoft.com/dotnet/runtime:7.0@sha256:8cd26108e7a329458e5b84342816ae39b2b38bd041cf  8.9s
 => => sha256:0908582d94f04fc7923e388bc582aa6fa1040f3b4df9674a73f842000db5ebeb 1.16kB / 1.16kB                  0.0s
 => => sha256:c95462acfb2e5a40ed2ae787ab293a230d7fdc362cfc87ce89c7a9a2440d11d5 1.91kB / 1.91kB                  0.0s
 => => sha256:728328ac3bde9b85225b1f0d60f5c149f5635a191f5d8eaeeb00e095d36ef9fd 31.43MB / 31.43MB                 2.1s
 => => sha256:82bb7a80de578404d92b5ae5e67f3de90eab30027694d2609be35ad25b09e3bc 14.97MB / 14.97MB                 1.8s
 => => sha256:534ba947de6ac79fd6168f4a93847954b23bab2782700bbfff7f31e61a03e8d4 32.46MB / 32.46MB                 2.8s
 => => sha256:f1b39e168c1c776458e172f157167607b9fd3cc550af8e6ff0a7dd363c1e64ea 153B / 153B                       2.2s
 => => extracting sha256:728328ac3bde9b85225b1f0d60f5c149f5635a191f5d8eaeeb00e095d36ef9fd                        4.5s
 => => extracting sha256:82bb7a80de578404d92b5ae5e67f3de90eab30027694d2609be35ad25b09e3bc                        1.0s
 => => extracting sha256:534ba947de6ac79fd6168f4a93847954b23bab2782700bbfff7f31e61a03e8d4                        0.6s
 => [internal] load build context                                                                                 0.2s
 => => transferring context: 7.13kB                                                                               0.0s
 => [build 1/7] FROM mcr.microsoft.com/dotnet/sdk:7.0@sha256:d32bd65cf5843f413e81f5d917057c82da99737cb1637e905a  8.9s
 => => resolve mcr.microsoft.com/dotnet/sdk:7.0@sha256:d32bd65cf5843f413e81f5d917057c82da99737cb1637e905a1a4bc2  0.1s
 => => sha256:d32bd65cf5843f413e81f5d917057c82da99737cb1637e905a1a4bc2e7ec6c8d 1.79kB / 1.79kB                   0.0s
 => => sha256:a291948a5e5ceb50e3eaa75cb1e6377660146b385e475938d6855e220afaaeed 2.01kB / 2.01kB                   0.0s
 => => sha256:ea4f5eec20952a885a89566fc35cf3295b3228375b715a0f1af9e5a3c0c2eebf 5.29kB / 5.29kB                   0.0s
 => => sha256:82bb7a80de578404d92b5ae5e67f3de90eab30027694d2609be35ad25b09e3bc 14.97MB / 14.97MB                 1.8s
 => => sha256:728328ac3bde9b85225b1f0d60f5c149f5635a191f5d8eaeeb00e095d36ef9fd 31.43MB / 31.43MB                 2.0s
 => => sha256:534ba947de6ac79fd6168f4a93847954b23bab2782700bbfff7f31e61a03e8d4 32.46MB / 32.46MB                 2.7s
 => => sha256:f1b39e168c1c776458e172f157167607b9fd3cc550af8e6ff0a7dd363c1e64ea 153B / 153B                       2.2s
 => => sha256:f194078e85f8008c084163778aaf266434f7182e0d4f783646f41b388c88a13a 10.12MB / 10.12MB                 3.5s
 => => sha256:22689bb63f95af71c8c0079742ff0afa570fa6cfc30ecfa7873976890829a24d 25.37MB / 25.37MB                 5.1s
 => => extracting sha256:728328ac3bde9b85225b1f0d60f5c149f5635a191f5d8eaeeb00e095d36ef9fd                        6.5s
```

Step 9: Run Vote Container (Frontend)

```
^Ccontrolplane:~$ docker run -d --name vote --network votingapp -p 8080:80   -e REDIS_HOST=redis -e REDIS_PORT=6379
 POSTGRES_HOST=db -e POSTGRES_PORT=5432   voteimage
0275207f6bd09d7efacda42cd215d8354c71d25016d7c6c560c17a77f16d3118
controlplane:~$
```

Step 10:- Run Result Container (Frontend)

Start the `result` service container, exposing it on port 8081 and setting Postgres connection environment variables:

```
controlplane:~$ docker run -d --name result1 --network votingapp -p 8081:80 -e POSTGRES_HOST=db -e POSTGRES_PORT=5432
-e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres resultimage
244b1c63e31921108741d89b763840af6dd24cbbfe79c4234b9932686eccb2d0
controlplane:~$
```

Step11:- Run Worker Container (Background Processor)

Run the worker service container connected to the network with environment variables for Redis and Postgres; no host port mapping is required:

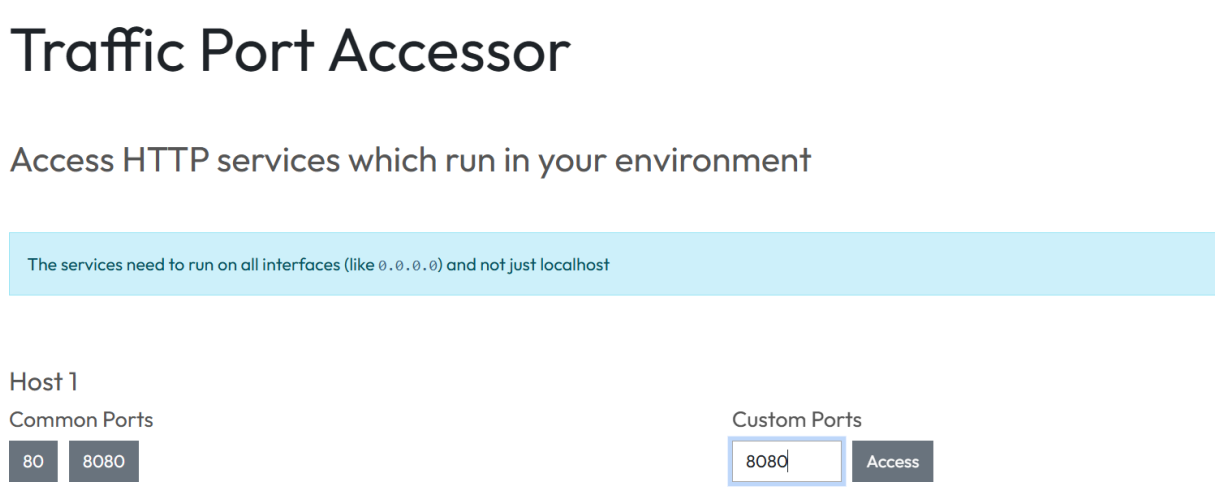
```
controlplane:~$ docker run -d --name worker1 --network votingapp  -e REDIS_HOST=redis -e REDIS_PORT=6379   -e POSTGRES
_HOST=db -e POSTGRES_PORT=5432   -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres   worker-img
0781516b41b809c6bf04970b36ae7aa3808bdabb93286667e86fba2c71a9d8be
controlplane:~$
```

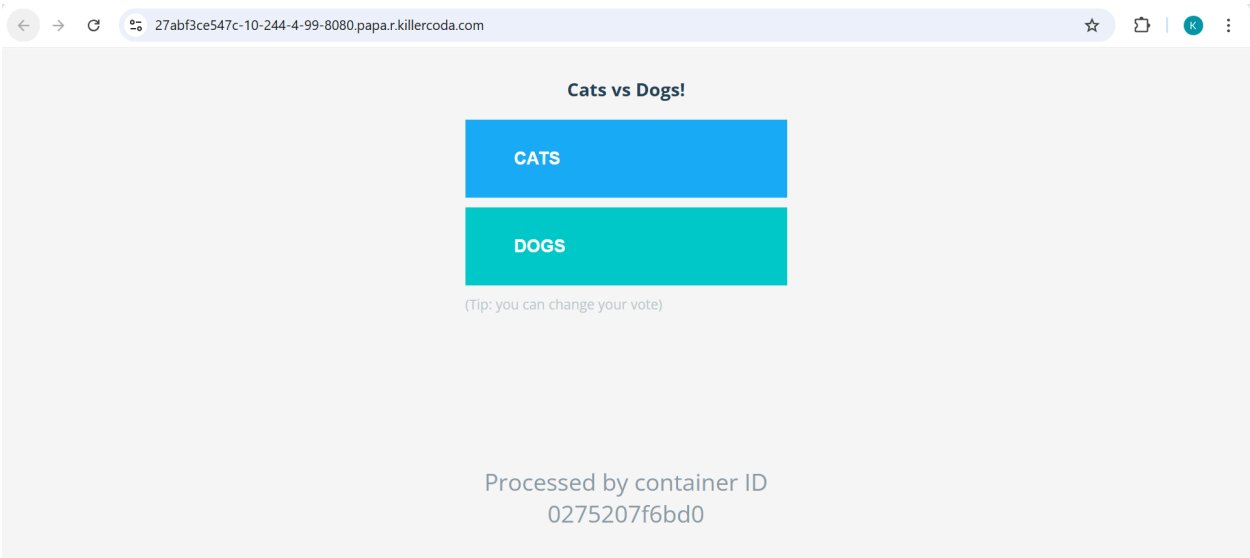
Step 12: Quick Status Checks

To list all running containers and verify that your services.

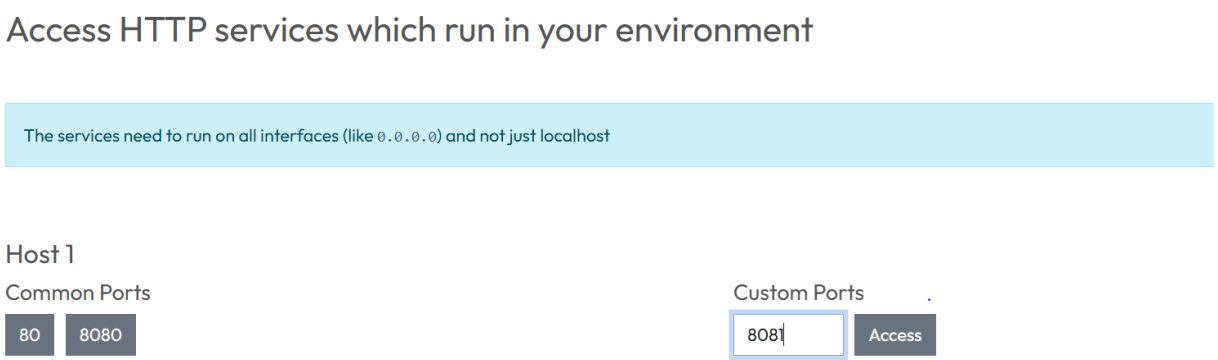
```
controlplane:~$ docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED              STATUS              PORTS
                                NAMES
6ab58da59f22   redis:alpine          "docker-entrypoint.s…"   4 seconds ago        Up 3 seconds        6379/tcp
                                redis1
d549d7892210   postgres:15-alpine    "docker-entrypoint.s…"   About a minute ago   Up About a minute   0.0.0.0:5432->54
32/tcp, [::]:5432->5432/tcp   db1
0781516b41b8   worker-img            "dotnet Worker.dll"      5 minutes ago        Up 5 minutes
                                worker1
244b1c63e319   resultimage           "/usr/bin/tini -- no…"   7 minutes ago        Up 7 minutes        0.0.0.0:8081->80
/tcp, [::]:8081->80/tcp       result1
0275207f6bd0   voteimage             "gunicorn app:app -b…"   12 minutes ago       Up 12 minutes       0.0.0.0:8080->80
/tcp, [::]:8080->80/tcp       vote
controlplane:~$
```

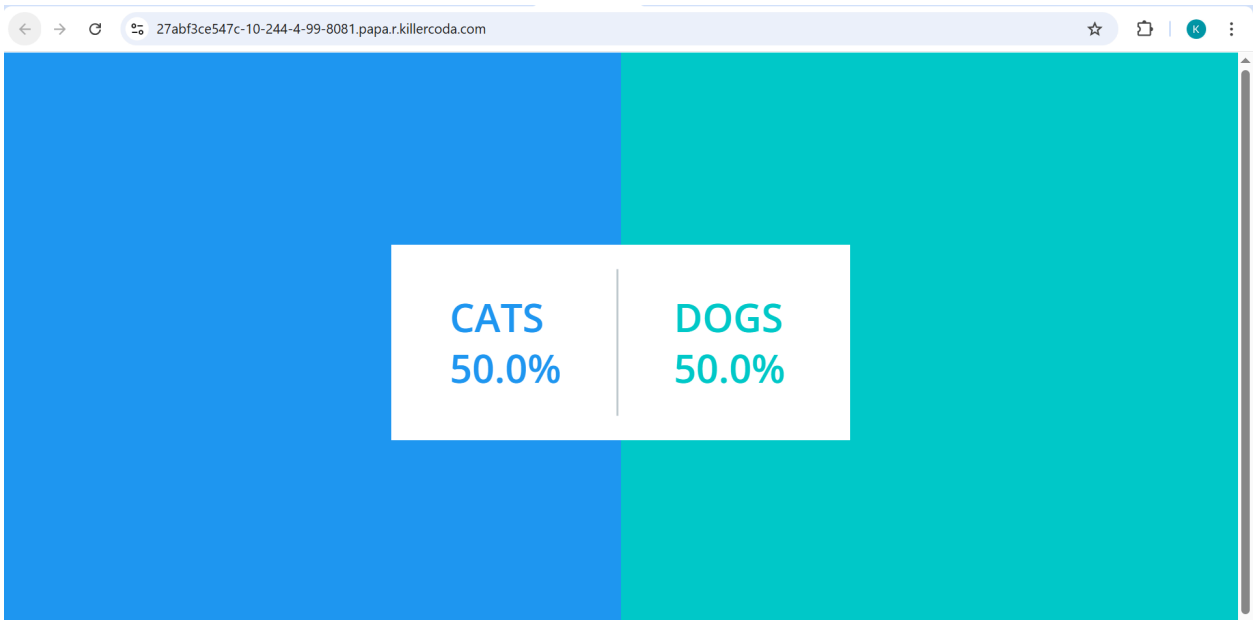
Step13:- You can now access the voting app by navigating to http://localhost:8080 in your web browser.





Step14:- You can now access the result app by navigating to http://localhost:8080 in your web browser.





THANK YOU!

-Yash Patil